

Evalueerbaar Onderdeel & Leerdoelen (Verticaal)	0 pt (Onbruikbaar)	1 pt (Onvoldoende)	2 pt (Bijna Voldoende)	3 pt (ECTS Basis - 50%)	4 pt (Gevorderd / Syllabus Bonus)	5 pt (Excellent / Syllabus Top)
1. Geheugenbeheer & Resource Management 4/20 <i>Gekoppelde leerdoelen:</i> 1.1 (Pointers/references) 1.2 (Allocatie & problemen)*	Project compileert niet of crasht direct bij runtime door grove geheugenfouten (segfaults).	Constance memory leaks (new zonder delete), dangling pointers of double frees.	Basis stack/heap is begrepen, maar bevat lekkages bij het herstarten van een level of entiteit-cleanup.	Handmatig geheugenbeheer is stabiel. new/delete zijn overal in balans. Geen actieve memory leaks.	Handmatig beheer is vermeden door consequent gebruik van smart pointers (unique_ptr/shared_ptr). = delete blokkeert foute kopieën.	Diep begrip van RAII. Circulaire afhankelijkheden worden correct opgevangen met std::weak_ptr. Waterdichte ownership-architectuur.
2. Type-hygiëne & OOP-Architectuur 4/20 <i>Gekoppelde leerdoelen:</i> 1.1 (Types) 4.1 (OOP-concepten)*	Geen OOP-structuur; alle data en statussen leven in globale variabelen of een gigantische main.	Foutief gebruik van overerving (wildgroei) of polymorfisme faalt door het ontbreken van virtual destructors.	Klassenstructuur werkt, maar encapsulatie ontbreekt volledig (bijna alle velden staan onnodig op public).	Correcte scheiding tussen struct (data) en class (encapsulatie via private). Runtime polymorfisme (virtual) klopt.	Strikte toepassing van const-correctness. Grote objecten gaan via const T&; read-only functies zijn expliciet const.	Statisch polymorfisme (templates) ingezet waar nuttig. Compositie verkozen boven te diepe overerving wegens cache-locality.
3. Collecties & Datamanagement 4/20 <i>Gekoppelde leerdoelen:</i> 1.1 (Arrays/strings) 4.2 (Basis algoritmes)*	Geen gebruik van STL-containers; hardcoded raw arrays die direct out-of-bounds crashen.	Game loop crasht bij cleanup van entiteiten wegens het muteren van een vector binnen een index-gebaseerde loop.	std::vector gebruikt, maar cleanup bevat foutgevoelige iterator-hacks. Alle leveledata staat hardcoded in code.	Entiteiten stabiel in std::vector. Opschonen tijdens game loop gebeurt crash-vrij via std::erase_if. Levels ingelezen via std::ifstream.	Inzet van ingebouwde STL-algoritmes (std::sort, std::transform) en std::map waar logisch. Robuuste I/O foutafhandeling.	Gebruik van C++20/23 Ranges en Views voor efficiënte lazy evaluation. Foutafhandeling via std::optional of std::expected.

Evalueerbaar Onderdeel & Leerdoelen (Verticaal)	0 pt (Onbruikbaar)	1 pt (Onvoldoende)	2 pt (Bijna Voldoende)	3 pt (ECTS Basis - 50%)	4 pt (Gevorderd / Syllabus Bonus)	5 pt (Excellent / Syllabus Top)
4. Unit Testing 3/20 <i>Gekoppelde leerdoelen:</i> 2.1 (Belang unit testing)*	Totaal geen unit tests of testbestanden aanwezig in het ingeleverde project.	Testbestanden bestaan in CMake, maar bevatten geen logische assertions of alle testen falen permanent.	Enkele basale runtime checks aanwezig, maar deze testen geen core gameplay- of data-logica.	Er zijn functionele unit tests geschreven die de core-logica (bijv. scoreberekening of collision-math) buiten de game-loop valideren.	Brede testdekking van verschillende scenario's en edge-cases, netjes geïntegreerd binnen de CMake/IDE test-suite.	Volledig testgedreven architectuur waarbij complexe logische componenten volledig geïsoleerd en geautomatiseerd getest worden.
5. Code-hygiëne & Projectstructuur 3/20 <i>Gekoppelde leerdoelen:</i> 3.1 (Debuggen) 4.2 (Applicatie bouwen)*	Cmake is onleesbaar door foutieve copy-pastes waardoor duidelijk blijkt dat hier weinig of geen kennis over bestaat.	Grove namespace pollution (using namespace std; in header-bestanden) of onleesbare spaghetti-code.	CMake werkt, maar rommelige header-hygiëne (missende guards) of #include-hacks om circulaire koppelingen te sussen.	Zuivere CMake-structuur. #pragma once/guards correct toegepast. Splitting header/source. Circulaire koppelingen opgelost via forward declarations.	Code is vrij van magische getallen. Foutloze header-hygiëne. Foutenopsporing gebeurt via de debugger in plaats van tijdelijke prints.	Modulaire en schone header-hygiëne die direct schaalbaar is voor grotere, professionele projecten (zoals Unreal Engine-standaarden).
6. Optimalisatie & Performance 2/20 <i>Gekoppelde leerdoelen:</i> 1.1 (Operators/types) 4.2 (Applicatie bouwen)*	Code bevat onnodig zware runtime-operaties in de update/draw loop die zware lag veroorzaken.	Zware object-kopieën (by-value) van textures of grote structs gebeuren continu onbewust binnen de real-time game loop.	Primitieve pogingen tot optimalisatie, maar inconsistent toegepast (bijv. herhaaldelijk herberekenen van statische data).	Efficiënt basisgebruik van types. Geen onnodige heap-allocaties (new) of zware overhead binnen de real-time update/render frames.	Waarden en constanten die vooraf bekend zijn, zijn strikt compile-time gedefinieerd via constexpr expressies om runtime overhead te elimineren.	Succesvolle en functionele inzet van bitwise operaties en bitmasking
Analyseren		Ontwerpen	Optimaliseren	Realiseren		
1,1 Verklaart fundamentele C++-concepten (types, operators, arrays, pointers, references en strings).		1,2 Verklaart hoe geheugenallocatie in C++ werkt en kan geheugenproblemen ontdekken.	2,1 Verklaart waarom unit testing belangrijk zijn.	3,1 Debugt C++ applicaties.	4,1 Kan object-geïntendeerde concepten toepassen in C++.	4,2 Ontwikkelt basis C++ algoritmes en applicaties.